# Optimization of the ZPIC Particle-in-Cell Code for the A64FX Vector Architecture

Renato Gonçalves
Department of Informatics
University of Minho
Braga, Portugal
pg61541@alunos.uminho.pt

Daniel Lobo
Department of Informatics
University of Minho
Braga, Portugal
pg60241@alunos.uminho.pt

***Abstract*— This paper presents the parallelization and optimization of a Particle-in-Cell (PIC) simulation code (zpic) targeting the Fujitsu A64FX architecture. Starting from a serial baseline of 76.30 seconds, we employed a structured methodology to identify hotspots and exploit thread-level and data-level parallelism using OpenMP. Key optimizations included SIMD vectorization for field solvers, NUMA-aware memory initialization to exploit the High Bandwidth Memory (HBM2), and a direct-atomic strategy for current deposition that leverages the A64FX's Large System Extension (LSE) hardware. We analyze the performance trade-offs between array privatization and atomic operations, as well as the impact of the A64FX's Core Memory Group (CMG) topology on scalability. The final optimized implementation achieves a simulation time of 13.86 seconds, representing a 5.51x speedup over the baseline.**

***Keywords*— *High Performance Computing, OpenMP, Particle-in-Cell, A64FX, Vectorization, Atomics.***

## I. Introduction

This Particle-in-Cell (PIC) codes are a cornerstone of computational plasma physics. They couple a particle pusher (Lagrangian step) with electromagnetic field solvers (Eulerian step) via current deposition and field interpolation. This dual nature presents distinct optimization challenges: field solvers are typically memory-bandwidth bound stencil operations, while particle pushers involve random memory access patterns prone to race conditions.

This project targets the optimization of the zpic educational code on the Deucalion cluster, specifically the A64FX processor. The A64FX is a 48-core ARM processor featuring Scalable Vector Extensions (SVE) with 512-bit vectors and high-bandwidth HBM2 memory. Crucially, the chip is divided into four Core Memory Groups (CMGs), essentially acting as a four-node NUMA system on a single die [1]. Understanding this topology is critical for performance, as cross-CMG data access incurs significant latency penalties.

## II. Profiling and Hotspot Analysis

Initial profiling of the serial application was performed using Score-P for instrumentation and CubeGUI for visualization. The analysis of serial execution revealed a total runtime of 76.30 seconds, distributed across significant hotspots:

### A. Particle Advance (spec_advance)

Occupied roughly 30% of the runtime. This cost was split between the relativistic particle movement (Boris pusher) and the computationally expensive current deposition, which suffers from high latency due to random memory access patterns.

### B. Field Advance (emf_advance)

Accounted for approximately 20% of the runtime, specifically localized within the Yee mesh solvers (yee_b and yee_e). These functions are characterized by low arithmetic intensity and are strictly memory-bandwidth bound.

### C. Current Smoothing (current_smooth)

This accounted for approximately 40% of the total execution time. Deeper analysis revealed this was dominated by the kernel_x function, which performs a convolution filter but suffered from heavy serial dependency chains.

Additionally, the profiling identified specific serial bottlenecks, such as *spec_sort* (particle sorting) and *spec_move_window*, which act as barriers preventing full scalability in a parallel environment.

## III. Optimization Methodology

### A. Memory Initialization and Vectorization

The first optimization phase addressed the First-Touch allocation policy. In the A64FX, memory pages are physically allocated on the CMG where they are first written. The original code used *memset* (serial execution), causing all memory to reside on CMG 0. We replaced this with parallel OpenMP loops during initialization, ensuring data was distributed across the four HBM2 stacks [2].

Simultaneously, we annotated the *yee_b* and *yee_e* loops with *#pragma omp parallel for simd*. We ensured 64-byte memory alignment using *posix_memalign* to maximize the efficiency of the 512-bit SVE load/store instructions..

### B. Handling Data Races in Current Deposition

The most critical challenge was parallelizing spec_advance. Particles moving in parallel may attempt to write to the same grid cell simultaneously (Current Deposition), creating a race condition. We analyzed three approaches to solve this:

*1) Critical Sections: Wrapping the deposition in #pragma omp critical. This proved inefficient due to massive serialization, resulting in minimal speedup.*

*2) Array Privatization (Reduction): Allocating private grids for each thread and summing them at the end of the timestep. While this eliminates race conditions, the A64FX suffered from severe cache thrashing during the reduction phase. The large memory stride required to gather data from*

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

*multiple thread buffers overwhelmed the Translation Lookaside Buffer (TLB).*

*3) Direct Atomics (Selected Approach): Threads write directly to the global grid using #pragma omp atomic update. While typically slow, the A64FX supports Large System Extensions (LSE), allowing atomic adds to be handled directly in the L2 cache hardware. Combined with particle sorting (performed every 10 iterations), cache line contention was minimized, making this the fastest approach.*

### C. The CMG Scalability Constraint

A distinct scalability limit was observed regarding the thread count. While the particle pusher scales well up to 48 cores, the field solvers (*emf*) and filters (*current*) degraded in performance when scaling beyond 12 threads.

- 1-12 Threads: Execution is contained within a single CMG (shared L2 cache). Performance scales linearly.
- 13-48 Threads: Execution spans multiple CMGs. For small grid sizes (20,000 cells), the simulation grid (~240KB) fits entirely in the L2 cache of one CMG. Spanning multiple CMGs forces data to traverse the on-chip ring bus, introducing latency that outweighs the computational gain of extra cores.

Consequently, the optimal configuration found was using 12 OpenMP threads, bound tightly to a single CMG using OMP_PROC_BIND=close. This "Hybrid Strategy" of 12 threads provided the best balance between compute power and memory latency.

### D. Current_smoothing

The optimization for the *Current Smoothing* hotspot (dominated by *kernel_x*) was achieved through parallelization and a data-race resolution technique:

- **Parallelization**: The 3-point convolution filter loop within kernel_x was parallelized using the #pragma omp parallel for directive.
- **Data Race Resolution (Buffer Swap)**: To enable parallel execution of the convolution without data races and to avoid serial dependency chains2, the logic was changed to use two buffers:
    - One buffer was used exclusively to read the input data (*J_buf_in*).
    - A second buffer was used exclusively to write the computational results (*J_buf_out*).
    - After the parallel loop completed, the current pointers (*current->J_buf* and *current->J_swap*) were immediately updated using a pointer-swapping technique. This avoids the memory and CPU overhead of a costly data copy (*memcpy*).

## IV. Experimental Results

### A. Performance Metrics

All tests were run on the Deucalion A64FX partition using GCC 13. The final configuration utilized 12 threads with atomic updates for particles and vectorized solvers.

Fig. 1. Comparison of runtime execution before and after optimization

| Function | Runtime | | |
|---|---|---|---|
| | ***Original (s)*** | ***Optimized (s)*** | ***Speedup*** |
| *spec_advance* | 46.77 | 8.72 | 5.36x |
| *emf_advance* | 11.13 | 3.35 | 3.32x |
| **Total Simulation** | **76.30** | **13.86** | **5.51x** |

### B. Discussion

The particle pusher achieved a 5.36x speedup on 12 threads. While ideal scaling would suggest higher gains, the random access nature of current deposition creates unavoidable L1 cache misses. The implementation of "persistent thread-local storage" prevented repeated *malloc* system calls inside the time loop, saving significant OS overhead.

The field solver speedup (3.32x) reflects the memory bandwidth saturation. Even with 12 cores, the arithmetic intensity of the Yee scheme is low enough that the HBM2 bandwidth becomes the limiting factor. Attempts to use 48 threads resulted in a solver time of ~8.1s (slower than serial), confirming the penalty of cross-CMG communication for small problem sizes.

## V. Conclusion

We successfully optimized the *zpic* code for the A64FX architecture, reducing execution time from 76.30s to 13.86s. The key insight was identifying the trade-off between parallelism and data locality. While the 48-core capacity suggests massive parallelism, the NUMA-like topology of the CMGs dictates that for small datasets, keeping data local to a single memory group (12 threads) yields superior performance to spanning the entire chip. Furthermore, modern ARM hardware atomics proved more efficient than software-based reduction techniques for handling data races in this specific workload.

## References


[1] Fujitsu, "A64FX Microarchitecture Manual," Fujitsu Limited, 2020. [Online]. Available: https://github.com/fujitsu/A64FX

[2] OpenMP Architecture Review Board, "OpenMP Application Program Interface Version 5.0," Nov. 2018. [Online]. Available: https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5.0.pdf